Understanding and preprocessing data

In [1]:
```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import precision_recall_curve, average_precision_score
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
from keras.layers import LSTM, GRU, RepeatVector, TimeDistributed, Dense
from keras.models import Sequential
from keras.layers import LSTM, RepeatVector, TimeDistributed, Dense
from sklearn.model_selection import train_test_split
from keras.models import Sequential
from keras.layers import GRU
from keras.layers import SimpleRNN
from sklearn.metrics import precision_recall_curve
```

In [2]:
```python
normal_file = 'E:\\UNIRI Projects\\Master Thesis topic\\x_raw_64.csv'
anomalous_file = 'E:\\UNIRI Projects\\Master Thesis topic\\x_raw_heavy_64.csv'
```

In [3]:
```python
normal_data = pd.read_csv(normal_file, header=None, names=['X', 'Y', 'Z'])
anomalous_data = pd.read_csv(anomalous_file, header=None, names=['X', 'Y', 'Z'])
```

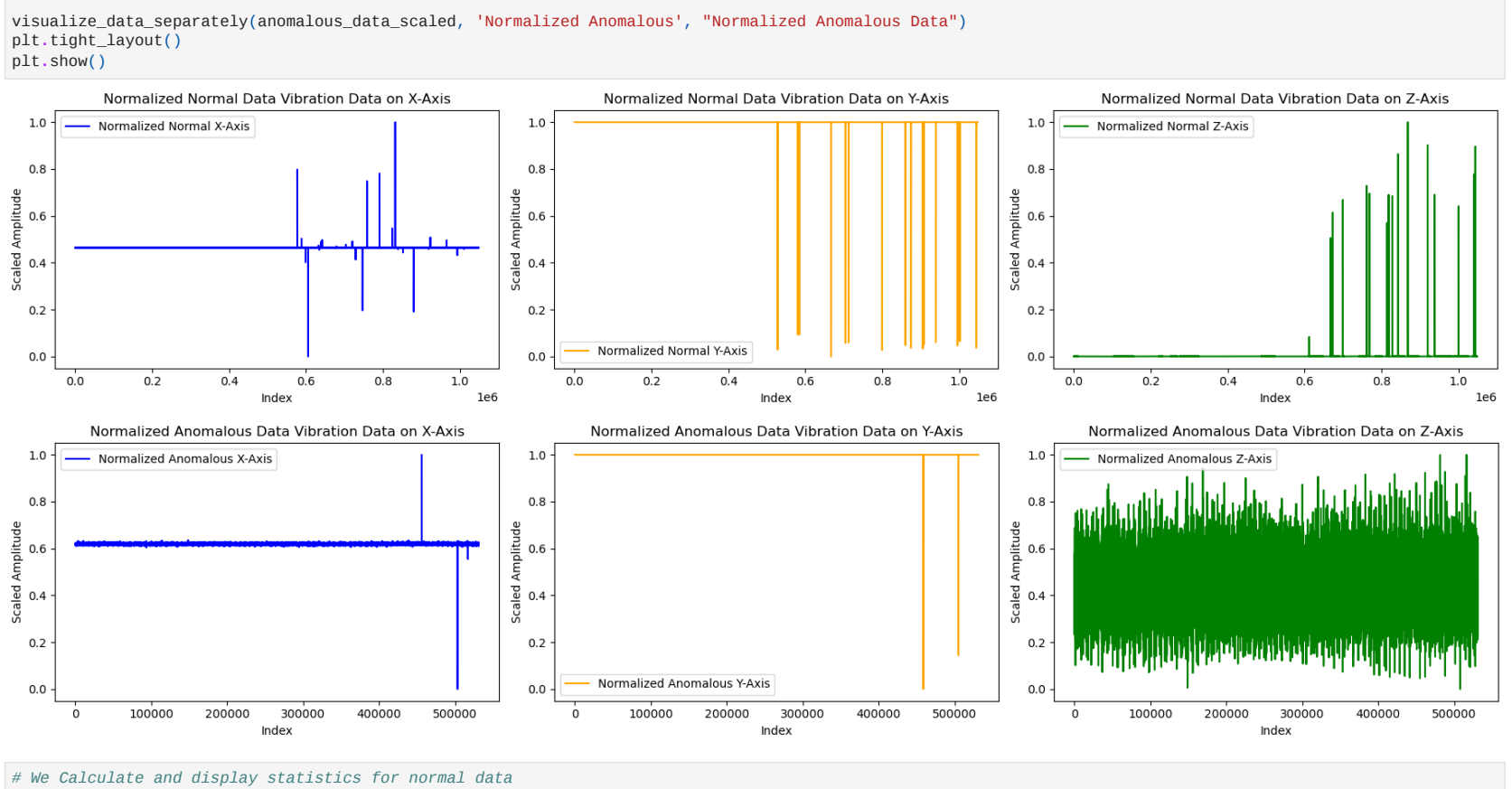
In [4]:
```python
print(normal_data.head(5))
print(normal_data.tail(5))
print(anomalous_data.head(5))
print(anomalous_data.tail(5))
```

```
     X      Y     Z
0  704 -16144  2336
1  320 -16496  2032
2 -768 -16416  1648
3 -592 -16560  2496
4  592 -15840  2816
           X      Y     Z
1048571  -224 -16512  1776
1048572   -16 -16800  1856
1048573  1072 -16592  2144
1048574    64 -16560  1328
1048575  -928 -15856  1488
     X      Y     Z
0 -608 -16112  1808
1 -272 -16512  1568
2 -288 -16544  1536
3 -656 -16400  1648
4 -656 -16480  1712
           X      Y     Z
531366 -2272 -15328  2160
531367  -592 -15440  2688
531368   416 -16688  2688
531369    96 -17648  1376
531370  1056 -17056  1024
```

In [5]:
```python
window_size = 5

normal_data_smooth = normal_data.rolling(window=window_size).mean().dropna()
anomalous_data_smooth = anomalous_data.rolling(window=window_size).mean().dropna()
```

In [6]:
```python
def min_max_scaler(data):
    return (data - data.min()) / (data.max() - data.min())

normal_data_scaled = min_max_scaler(normal_data_smooth)
anomalous_data_scaled = min_max_scaler(anomalous_data_smooth)
```

In [7]:
```python
def visualize_data_separately(data, data_label, title_prefix):
    axes = data.columns
    colors = ['blue', 'orange', 'green']
    fig, axs = plt.subplots(nrows=1, ncols=len(axes), figsize=(18, 4))

    for i, axis in enumerate(axes):
        axs[i].plot(data.index, data[axis], label=f'{data_label} {axis}-Axis', color=colors[i])
        axs[i].set_title(f"{title_prefix} Vibration Data on {axis}-Axis")
        axs[i].set_xlabel("Index")
        axs[i].set_ylabel("Scaled Amplitude")
        axs[i].legend()

# Visualize the scaled data for each axis separately
visualize_data_separately(normal_data_scaled, 'Normalized Normal', "Normalized Normal Data")
plt.tight_layout()
plt.show()

visualize_data_separately(anomalous_data_scaled, 'Normalized Anomalous', "Normalized Anomalous Data")
plt.tight_layout()
plt.show()
```



In [8]:
```python
# We Calculate and display statistics for normal data
normal_data_statistics = normal_data.describe().T  # .T is to transpose the result for better readability
normal_data_statistics['skew'] = normal_data.skew()
normal_data_statistics['kurt'] = normal_data.kurtosis()

# We Calculate and display statistics for anomalous data
anomalous_data_statistics = anomalous_data.describe().T
anomalous_data_statistics['skew'] = anomalous_data.skew()
anomalous_data_statistics['kurt'] = anomalous_data.kurtosis()

print("Statistics for Normal Data:")
print(normal_data_statistics)
print("\nStatistics for Anomalous Data:")
print(anomalous_data_statistics)
```

```
Statistics for Normal Data:
       count          mean           std           min      25%      50%  \
X  1048576.0     54.372116  2.808795e+04 -1.360213e+07   -464.0     32.0
Y  1048576.0 -39947.078979  6.232925e+06 -1.732816e+09 -16672.0 -16384.0
Z  1048576.0   2231.130356  8.247010e+04 -4.960000e+03   1664.0   1920.0

       75%         max        skew           kurt
X    544.0  15684304.0  114.589632  189966.999454
Y -16096.0     17088.0 -264.631579   70070.258483
Z   2176.0  29442784.0  276.838761   79028.594844

Statistics for Anomalous Data:
      count          mean           std           min      25%      50%  \
X  531371.0     80.229143  3.011402e+03 -1.616816e+06   -656.0     80.0
Y  531371.0 -22015.697868  2.920450e+06 -1.617616e+09 -16640.0 -16368.0
Z  531371.0   1945.436059  7.701011e+02 -7.472000e+03   1584.0   1936.0

       75%       max        skew           kurt
X    832.0  992144.0 -224.358994  178592.455281
Y -16096.0   15968.0 -520.079109  272042.553260
Z   2304.0   11920.0    0.072557       5.712747
```

Divide the normalized data into sequences to serve as inputs to the LSTM autoencoder.

In [9]:
```python
# We can create a Function for sequences of data for the LSTM
def create_sequences(data, window_size):
    sequences = []
    for i in range(len(data) - window_size + 1):
        sequence = data.iloc[i:(i + window_size)].values
        sequences.append(sequence)
    return np.array(sequences)

# We can Prepare the input data as sequences
X = create_sequences(normal_data_scaled, window_size)
```

Design and construct the LSTM autoencoder with specified architecture and hyperparameters.

In [10]:
```python
input_dim = X.shape[2]  # number of features ('X', 'Y', 'Z'), so 3

# We can Split the data into training and testing sets
X_train, X_test = train_test_split(X, test_size=0.2, random_state=42)
```

In [11]:
```python
# Defination of LSTM autoencoder
def build_lstm_autoencoder(input_dim, timesteps, latent_dim=50):
    model = Sequential()

    # Encoder
    model.add(LSTM(100, activation='relu', input_shape=(timesteps, input_dim), return_sequences=True))
    model.add(LSTM(50, activation='relu', return_sequences=False))
    model.add(RepeatVector(timesteps))  # Set the dimensionality of the output space

    # Decoder
    model.add(LSTM(50, activation='relu', return_sequences=True))
    model.add(LSTM(100, activation='relu', return_sequences=True))
    model.add(TimeDistributed(Dense(input_dim)))

    model.compile(optimizer='adam', loss='mse')
    return model
```

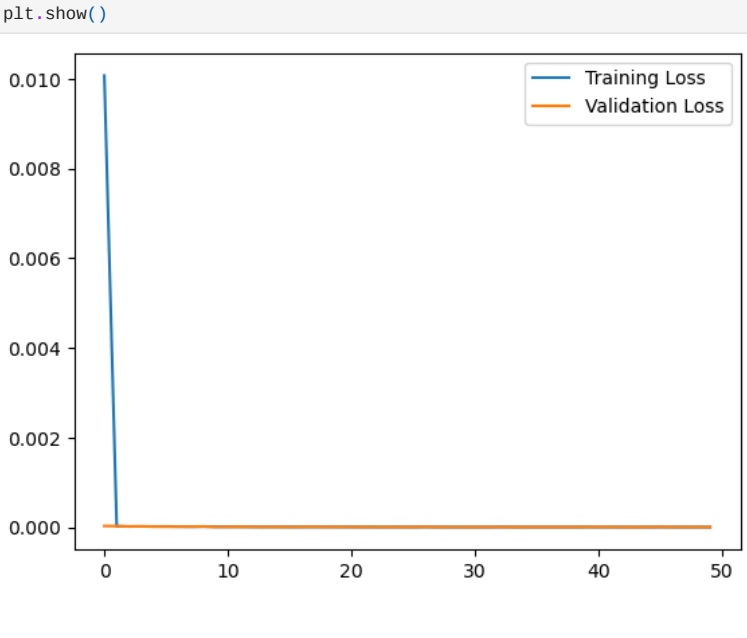
Train the LSTM autoencoder using the sequenced normal data and validate

In [12]:
```python
# We can Build the autoencoder
autoencoder = build_lstm_autoencoder(input_dim=input_dim, timesteps=window_size)

# We can Train the autoencoder
history = autoencoder.fit(X_train, X_train, epochs=50, batch_size=512, validation_split=0.2, verbose=1)
```

```
Epoch 1/50
1311/1311 [==============================] - 154s 112ms/step - loss: 0.0101 - val_loss: 2.6650e-05
Epoch 2/50
1311/1311 [==============================] - 123s 94ms/step - loss: 2.0660e-05 - val_loss: 2.2673e-05
Epoch 3/50
1311/1311 [==============================] - 128s 98ms/step - loss: 1.5986e-05 - val_loss: 1.4072e-05
Epoch 4/50
1311/1311 [==============================] - 144s 110ms/step - loss: 1.6466e-05 - val_loss: 1.8613e-05
Epoch 5/50
1311/1311 [==============================] - 170s 129ms/step - loss: 1.5123e-05 - val_loss: 1.1471e-05
Epoch 6/50
1311/1311 [==============================] - 141s 107ms/step - loss: 1.5460e-05 - val_loss: 1.0118e-05
Epoch 7/50
1311/1311 [==============================] - 157s 120ms/step - loss: 1.2398e-05 - val_loss: 1.1794e-05
Epoch 8/50
1311/1311 [==============================] - 178s 136ms/step - loss: 9.9277e-06 - val_loss: 1.1616e-05
Epoch 9/50
1311/1311 [==============================] - 178s 136ms/step - loss: 1.1278e-05 - val_loss: 1.6509e-05
Epoch 10/50
1311/1311 [==============================] - 158s 120ms/step - loss: 8.6865e-06 - val_loss: 5.8916e-06
Epoch 11/50
1311/1311 [==============================] - 147s 112ms/step - loss: 7.5503e-06 - val_loss: 6.4619e-06
Epoch 12/50
1311/1311 [==============================] - 146s 112ms/step - loss: 7.0318e-06 - val_loss: 9.2031e-06
Epoch 13/50
1311/1311 [==============================] - 149s 113ms/step - loss: 6.1514e-06 - val_loss: 6.2708e-06
Epoch 14/50
1311/1311 [==============================] - 146s 112ms/step - loss: 5.1389e-06 - val_loss: 4.6826e-06
Epoch 15/50
1311/1311 [==============================] - 147s 112ms/step - loss: 5.7151e-06 - val_loss: 5.3055e-06
Epoch 16/50
1311/1311 [==============================] - 136s 104ms/step - loss: 4.5568e-06 - val_loss: 3.6133e-06
Epoch 17/50
1311/1311 [==============================] - 147s 112ms/step - loss: 4.6255e-06 - val_loss: 6.1333e-06
Epoch 18/50
1311/1311 [==============================] - 149s 114ms/step - loss: 4.1739e-06 - val_loss: 8.0961e-06
Epoch 19/50
1311/1311 [==============================] - 150s 114ms/step - loss: 4.3860e-06 - val_loss: 4.0571e-06
Epoch 20/50
1311/1311 [==============================] - 149s 113ms/step - loss: 3.9326e-06 - val_loss: 6.2509e-06
Epoch 21/50
1311/1311 [==============================] - 150s 114ms/step - loss: 4.0577e-06 - val_loss: 2.9172e-06
Epoch 22/50
1311/1311 [==============================] - 147s 112ms/step - loss: 3.3403e-06 - val_loss: 3.0596e-06
Epoch 23/50
1311/1311 [==============================] - 148s 113ms/step - loss: 3.3594e-06 - val_loss: 2.6896e-06
Epoch 24/50
1311/1311 [==============================] - 148s 113ms/step - loss: 3.1486e-06 - val_loss: 2.2989e-06
Epoch 25/50
1311/1311 [==============================] - 148s 113ms/step - loss: 2.6111e-06 - val_loss: 3.4973e-06
Epoch 26/50
1311/1311 [==============================] - 148s 113ms/step - loss: 2.4542e-06 - val_loss: 2.1723e-06
Epoch 27/50
1311/1311 [==============================] - 148s 113ms/step - loss: 3.1460e-06 - val_loss: 6.5799e-06
Epoch 28/50
1311/1311 [==============================] - 148s 113ms/step - loss: 2.4587e-06 - val_loss: 2.0849e-06
Epoch 29/50
1311/1311 [==============================] - 148s 113ms/step - loss: 1.8760e-06 - val_loss: 1.4459e-06
Epoch 30/50
1311/1311 [==============================] - 147s 112ms/step - loss: 1.4461e-06 - val_loss: 2.2009e-06
Epoch 31/50
1311/1311 [==============================] - 151s 115ms/step - loss: 2.8101e-06 - val_loss: 1.6090e-06
Epoch 32/50
1311/1311 [==============================] - 149s 114ms/step - loss: 1.5348e-06 - val_loss: 1.6792e-06
Epoch 33/50
1311/1311 [==============================] - 149s 114ms/step - loss: 3.8587e-06 - val_loss: 4.1573e-06
Epoch 34/50
1311/1311 [==============================] - 150s 114ms/step - loss: 2.6939e-06 - val_loss: 1.5267e-06
Epoch 35/50
1311/1311 [==============================] - 149s 114ms/step - loss: 1.9852e-06 - val_loss: 1.6272e-06
Epoch 36/50
1311/1311 [==============================] - 149s 114ms/step - loss: 1.7076e-06 - val_loss: 1.4298e-06
Epoch 37/50
1311/1311 [==============================] - 149s 114ms/step - loss: 1.3531e-06 - val_loss: 1.3247e-06
Epoch 38/50
1311/1311 [==============================] - 149s 114ms/step - loss: 1.5976e-06 - val_loss: 1.6413e-06
Epoch 39/50
1311/1311 [==============================] - 148s 113ms/step - loss: 1.6757e-06 - val_loss: 1.0712e-06
Epoch 40/50
1311/1311 [==============================] - 146s 112ms/step - loss: 2.2947e-06 - val_loss: 5.2541e-06
Epoch 41/50
1311/1311 [==============================] - 147s 112ms/step - loss: 2.1119e-06 - val_loss: 3.8795e-06
Epoch 42/50
1311/1311 [==============================] - 154s 117ms/step - loss: 1.5987e-06 - val_loss: 3.5681e-06
Epoch 43/50
1311/1311 [==============================] - 150s 114ms/step - loss: 1.2057e-06 - val_loss: 1.3672e-06
Epoch 44/50
1311/1311 [==============================] - 149s 114ms/step - loss: 1.1287e-06 - val_loss: 9.5966e-07
Epoch 45/50
1311/1311 [==============================] - 146s 111ms/step - loss: 1.5706e-06 - val_loss: 1.7804e-06
Epoch 46/50
1311/1311 [==============================] - 146s 111ms/step - loss: 1.7464e-06 - val_loss: 7.3617e-06
Epoch 47/50
1311/1311 [==============================] - 148s 113ms/step - loss: 9.8684e-07 - val_loss: 1.0010e-06
Epoch 48/50
1311/1311 [==============================] - 145s 111ms/step - loss: 1.3455e-06 - val_loss: 2.3124e-06
Epoch 49/50
1311/1311 [==============================] - 147s 112ms/step - loss: 1.1338e-06 - val_loss: 1.7604e-06
Epoch 50/50
1311/1311 [==============================] - 148s 113ms/step - loss: 8.6925e-07 - val_loss: 9.8283e-07
```

Analyze the training and validation losses to assess the model's learning process.

In [13]:
```python
# We can observe the training and validation loss
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.legend()
plt.show()
```



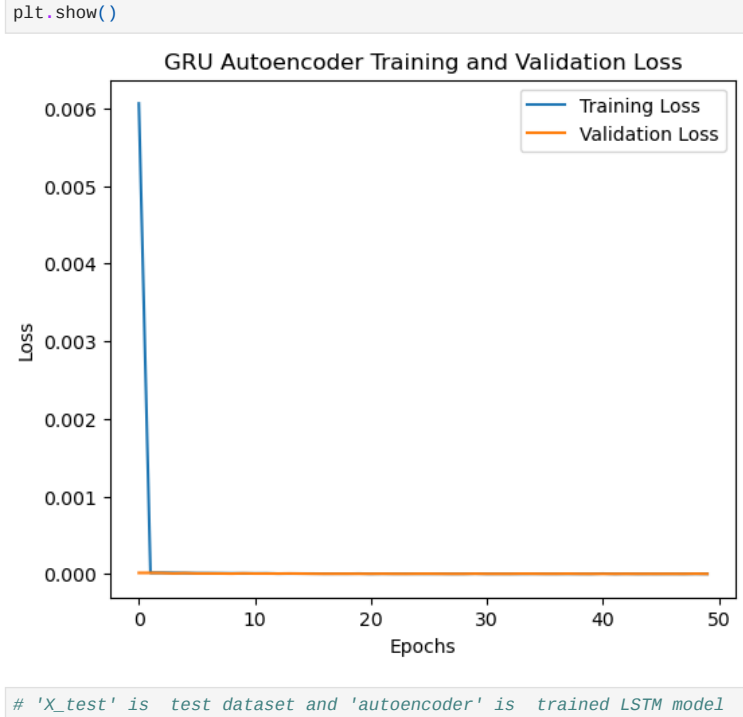
Comparing GRU Performance against LSTM

In [14]:
```python
# We are Defining the GRU autoencoder for comparison
def build_gru_autoencoder(input_dim, timesteps, latent_dim=50):
    model = Sequential()

    # Encoder
    model.add(GRU(100, activation='relu', input_shape=(timesteps, input_dim), return_sequences=True))
    model.add(GRU(latent_dim, activation='relu', return_sequences=False))
    model.add(RepeatVector(timesteps))

    # Decoder
    model.add(GRU(latent_dim, activation='relu', return_sequences=True))
    model.add(GRU(100, activation='relu', return_sequences=True))
    model.add(TimeDistributed(Dense(input_dim)))

    model.compile(optimizer='adam', loss='mse')
    return model

# We are Building and train GRU autoencoder
gru_autoencoder = build_gru_autoencoder(input_dim, window_size)
gru_history = gru_autoencoder.fit(X_train, X_train, epochs=50, batch_size=512, validation_split=0.2, verbose=1)
```

```
Epoch 1/50
1311/1311 [==============================] - 110s 80ms/step - loss: 0.0061 - val_loss: 1.3380e-05
Epoch 2/50
1311/1311 [==============================] - 105s 80ms/step - loss: 1.3806e-05 - val_loss: 1.3477e-05
Epoch 3/50
1311/1311 [==============================] - 105s 80ms/step - loss: 1.4528e-05 - val_loss: 1.2161e-05
Epoch 4/50
1311/1311 [==============================] - 106s 81ms/step - loss: 1.3439e-05 - val_loss: 6.9112e-06
Epoch 5/50
1311/1311 [==============================] - 105s 80ms/step - loss: 1.0986e-05 - val_loss: 1.0671e-05
Epoch 6/50
1311/1311 [==============================] - 114s 87ms/step - loss: 9.4502e-06 - val_loss: 6.2452e-06
Epoch 7/50
1311/1311 [==============================] - 118s 90ms/step - loss: 9.0507e-06 - val_loss: 6.0951e-06
Epoch 8/50
1311/1311 [==============================] - 112s 85ms/step - loss: 8.1531e-06 - val_loss: 5.4581e-06
Epoch 9/50
1311/1311 [==============================] - 138s 105ms/step - loss: 7.4070e-06 - val_loss: 3.7518e-06
Epoch 10/50
1311/1311 [==============================] - 134s 102ms/step - loss: 6.7959e-06 - val_loss: 7.2509e-06
Epoch 11/50
1311/1311 [==============================] - 138s 106ms/step - loss: 6.2953e-06 - val_loss: 4.7580e-06
Epoch 12/50
1311/1311 [==============================] - 121s 93ms/step - loss: 6.8802e-06 - val_loss: 4.7020e-06
Epoch 13/50
1311/1311 [==============================] - 128s 98ms/step - loss: 3.9207e-06 - val_loss: 4.2109e-06
Epoch 14/50
1311/1311 [==============================] - 108s 82ms/step - loss: 4.4137e-06 - val_loss: 5.5459e-06
Epoch 15/50
1311/1311 [==============================] - 107s 81ms/step - loss: 4.5168e-06 - val_loss: 3.9802e-06
Epoch 16/50
1311/1311 [==============================] - 106s 81ms/step - loss: 4.1520e-06 - val_loss: 2.3369e-06
Epoch 17/50
1311/1311 [==============================] - 108s 82ms/step - loss: 3.1543e-06 - val_loss: 1.6845e-06
Epoch 18/50
1311/1311 [==============================] - 110s 84ms/step - loss: 3.3346e-06 - val_loss: 1.8504e-06
Epoch 19/50
1311/1311 [==============================] - 107s 82ms/step - loss: 3.0908e-06 - val_loss: 2.3260e-06
Epoch 20/50
1311/1311 [==============================] - 108s 82ms/step - loss: 4.5751e-06 - val_loss: 1.9384e-06
Epoch 21/50
1311/1311 [==============================] - 109s 83ms/step - loss: 1.7635e-06 - val_loss: 1.3924e-06
Epoch 22/50
1311/1311 [==============================] - 110s 84ms/step - loss: 2.9549e-06 - val_loss: 2.4212e-06
Epoch 23/50
1311/1311 [==============================] - 109s 83ms/step - loss: 2.2315e-06 - val_loss: 1.2844e-06
Epoch 24/50
1311/1311 [==============================] - 109s 83ms/step - loss: 2.6384e-06 - val_loss: 8.0587e-07
Epoch 25/50
1311/1311 [==============================] - 109s 83ms/step - loss: 2.1145e-06 - val_loss: 2.2664e-06
Epoch 26/50
1311/1311 [==============================] - 112s 85ms/step - loss: 1.7311e-06 - val_loss: 2.7374e-06
Epoch 27/50
1311/1311 [==============================] - 109s 83ms/step - loss: 2.3851e-06 - val_loss: 1.2400e-06
Epoch 28/50
1311/1311 [==============================] - 110s 84ms/step - loss: 1.2591e-06 - val_loss: 7.8882e-07
Epoch 29/50
1311/1311 [==============================] - 109s 83ms/step - loss: 1.4092e-06 - val_loss: 9.4386e-07
Epoch 30/50
1311/1311 [==============================] - 110s 84ms/step - loss: 3.4883e-06 - val_loss: 2.7993e-06
Epoch 31/50
1311/1311 [==============================] - 109s 83ms/step - loss: 1.2840e-06 - val_loss: 9.9568e-07
Epoch 32/50
1311/1311 [==============================] - 109s 83ms/step - loss: 1.5247e-06 - val_loss: 8.9791e-07
Epoch 33/50
1311/1311 [==============================] - 110s 84ms/step - loss: 1.2120e-06 - val_loss: 1.0234e-06
Epoch 34/50
1311/1311 [==============================] - 112s 86ms/step - loss: 1.1915e-06 - val_loss: 3.0902e-06
Epoch 35/50
1311/1311 [==============================] - 110s 84ms/step - loss: 2.0469e-06 - val_loss: 2.3746e-06
Epoch 36/50
1311/1311 [==============================] - 112s 85ms/step - loss: 1.5710e-06 - val_loss: 1.2339e-06
Epoch 37/50
1311/1311 [==============================] - 112s 85ms/step - loss: 1.1954e-06 - val_loss: 1.4122e-06
Epoch 38/50
1311/1311 [==============================] - 112s 85ms/step - loss: 1.2621e-06 - val_loss: 3.0498e-06
Epoch 39/50
1311/1311 [==============================] - 112s 86ms/step - loss: 1.0863e-06 - val_loss: 9.8267e-07
Epoch 40/50
1311/1311 [==============================] - 112s 85ms/step - loss: 9.2901e-07 - val_loss: 6.2437e-07
Epoch 41/50
1311/1311 [==============================] - 113s 86ms/step - loss: 4.3050e-06 - val_loss: 9.8693e-07
Epoch 42/50
1311/1311 [==============================] - 121s 92ms/step - loss: 8.7773e-07 - val_loss: 7.1058e-07
Epoch 43/50
1311/1311 [==============================] - 106s 81ms/step - loss: 2.0124e-06 - val_loss: 1.2034e-06
Epoch 44/50
1311/1311 [==============================] - 104s 79ms/step - loss: 7.5370e-07 - val_loss: 7.1130e-07
Epoch 45/50
1311/1311 [==============================] - 105s 80ms/step - loss: 9.4456e-07 - val_loss: 7.6743e-07
Epoch 46/50
1311/1311 [==============================] - 106s 81ms/step - loss: 7.5584e-07 - val_loss: 9.2820e-07
Epoch 47/50
1311/1311 [==============================] - 104s 80ms/step - loss: 1.0861e-06 - val_loss: 8.4946e-07
Epoch 48/50
1311/1311 [==============================] - 105s 80ms/step - loss: 6.9849e-07 - val_loss: 6.6194e-07
Epoch 49/50
1311/1311 [==============================] - 104s 80ms/step - loss: 1.4474e-06 - val_loss: 2.7575e-06
Epoch 50/50
1311/1311 [==============================] - 104s 79ms/step - loss: 7.9569e-07 - val_loss: 6.5269e-07
```

Analyze the training and validation losses to assess the model's learning process.

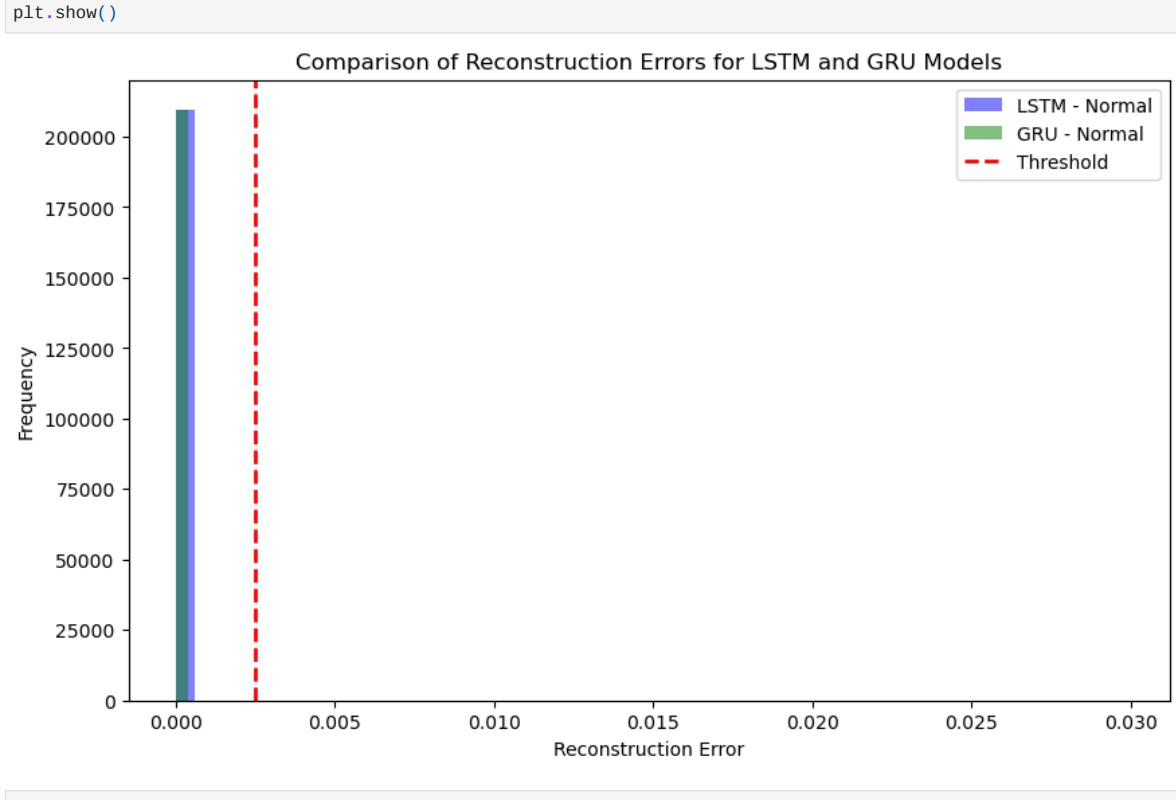
In [50]:
```python
plt.figure(figsize=(6, 5))
plt.plot(gru_history.history['loss'], label='Training Loss')
plt.plot(gru_history.history['val_loss'], label='Validation Loss')
plt.title('GRU Autoencoder Training and Validation Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.show()
```



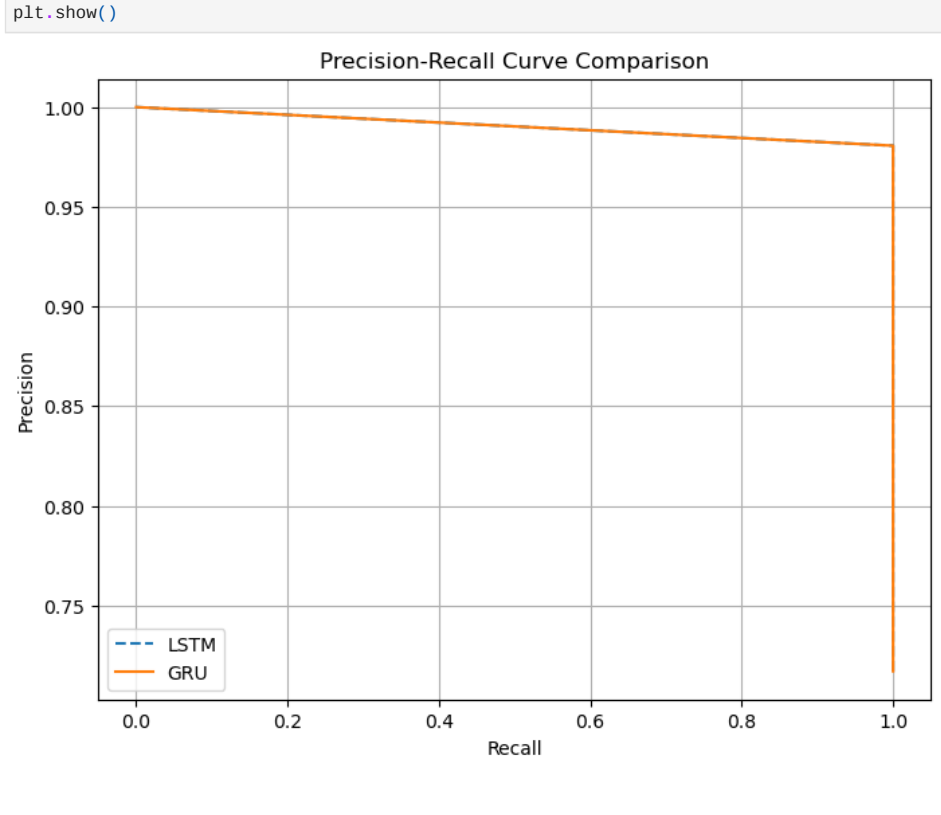
In [30]:
```python
# 'X_test' is  test dataset and 'autoencoder' is  trained LSTM model
# we are Predicting the reconstruction from the LSTM model
lstm_predictions = autoencoder.predict(X_test)
lstm_reconstruction_error = np.mean(np.power(X_test - lstm_predictions, 2), axis=(1, 2))

#  'gru_autoencoder' is  trained GRU model
# Predicting the reconstruction from the GRU model
gru_predictions = gru_autoencoder.predict(X_test)
gru_reconstruction_error = np.mean(np.power(X_test - gru_predictions, 2), axis=(1, 2))

# Define the anomaly detection threshold as the 95th percentile of the LSTM reconstruction error
threshold = np.percentile(lstm_reconstruction_error, 95)
```

```
6554/6554 [==============================] - 35s 5ms/step
6554/6554 [==============================] - 34s 5ms/step
```

In [83]:
```python
# We are Plotting the histograms
plt.figure(figsize=(10, 6))
plt.hist(lstm_reconstruction_error, bins=50, color='blue', alpha=0.5, label='LSTM - Normal')
plt.hist(gru_reconstruction_error, bins=50, color='green', alpha=0.5, label='GRU - Normal')
plt.axvline(threshold, color='red', linestyle='dashed', linewidth=2, label='Threshold')
plt.title('Comparison of Reconstruction Errors for LSTM and GRU Models')
plt.xlabel('Reconstruction Error')
plt.ylabel('Frequency')
plt.legend()
plt.show()
```



In [191…
```python
precision_lstm, recall_lstm, _ = precision_recall_curve(y_true, y_pred_lstm)
precision_gru, recall_gru, _ = precision_recall_curve(y_true, y_pred_gru)

#  precision-recall curves.
plt.figure(figsize=(8, 6))
plt.plot(recall_lstm, precision_lstm, label='LSTM', linestyle='--')
plt.plot(recall_gru, precision_gru, label='GRU', linestyle='-')
plt.xlabel('Recall')
plt.ylabel('Precision')
plt.title('Precision-Recall Curve Comparison')
plt.legend()
plt.grid()
plt.show()
```



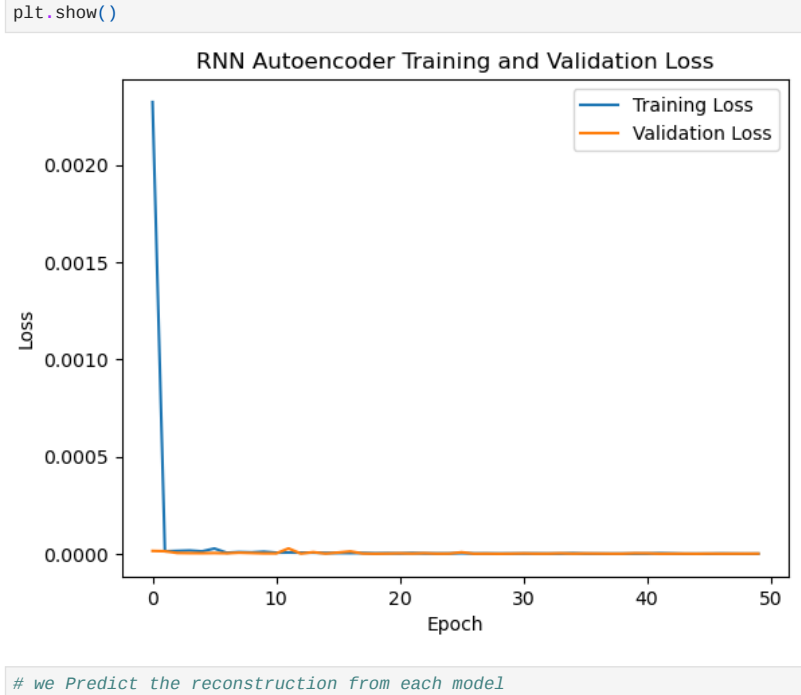
Again Comparing RNN Performance Against LSTM and GRU

In [56]:
```python
# RNN autoencoder
def build_rnn_autoencoder(input_dim, timesteps, latent_dim=50):
    model = Sequential()

    # Encoder
    model.add(SimpleRNN(100, activation='relu', input_shape=(timesteps, input_dim), return_sequences=True))
    model.add(SimpleRNN(latent_dim, activation='relu', return_sequences=False))
    model.add(RepeatVector(timesteps))

    # Decoder
    model.add(SimpleRNN(latent_dim, activation='relu', return_sequences=True))
    model.add(SimpleRNN(100, activation='relu', return_sequences=True))
    model.add(TimeDistributed(Dense(input_dim)))

    model.compile(optimizer='adam', loss='mse')
    return model

# we Build and train RNN autoencoder
rnn_autoencoder = build_rnn_autoencoder(input_dim, window_size)
rnn_history = rnn_autoencoder.fit(X_train, X_train, epochs=50, batch_size=512, validation_split=0.2, verbose=1)
```

```
Epoch 1/50
1311/1311 [==============================] - 47s 33ms/step - loss: 0.0023 - val_loss: 1.5205e-05
Epoch 2/50
1311/1311 [==============================] - 42s 32ms/step - loss: 1.3424e-05 - val_loss: 1.3715e-05
Epoch 3/50
1311/1311 [==============================] - 41s 31ms/step - loss: 1.5411e-05 - val_loss: 4.7242e-06
Epoch 4/50
1311/1311 [==============================] - 40s 31ms/step - loss: 1.7396e-05 - val_loss: 3.9582e-06
Epoch 5/50
1311/1311 [==============================] - 43s 33ms/step - loss: 1.3086e-05 - val_loss: 3.5389e-06
Epoch 6/50
1311/1311 [==============================] - 45s 34ms/step - loss: 2.7517e-05 - val_loss: 4.3500e-06
Epoch 7/50
1311/1311 [==============================] - 39s 30ms/step - loss: 5.9926e-06 - val_loss: 3.1001e-06
Epoch 8/50
1311/1311 [==============================] - 38s 29ms/step - loss: 9.3345e-06 - val_loss: 6.0822e-06
Epoch 9/50
1311/1311 [==============================] - 37s 28ms/step - loss: 7.8343e-06 - val_loss: 4.3521e-06
Epoch 10/50
1311/1311 [==============================] - 37s 28ms/step - loss: 1.1419e-05 - val_loss: 2.6505e-06
Epoch 11/50
1311/1311 [==============================] - 43s 33ms/step - loss: 6.4125e-06 - val_loss: 2.5114e-06
Epoch 12/50
1311/1311 [==============================] - 44s 33ms/step - loss: 7.0793e-06 - val_loss: 2.7957e-05
Epoch 13/50
1311/1311 [==============================] - 43s 32ms/step - loss: 6.4071e-06 - val_loss: 1.5105e-06
Epoch 14/50
1311/1311 [==============================] - 43s 33ms/step - loss: 6.5747e-06 - val_loss: 9.0233e-06
Epoch 15/50
1311/1311 [==============================] - 46s 35ms/step - loss: 4.9620e-06 - val_loss: 1.4576e-06
Epoch 16/50
1311/1311 [==============================] - 47s 36ms/step - loss: 4.3315e-06 - val_loss: 6.9054e-06
Epoch 17/50
1311/1311 [==============================] - 43s 33ms/step - loss: 3.9904e-06 - val_loss: 1.2950e-05
Epoch 18/50
1311/1311 [==============================] - 43s 33ms/step - loss: 4.4776e-06 - val_loss: 2.0476e-06
Epoch 19/50
1311/1311 [==============================] - 41s 31ms/step - loss: 3.1508e-06 - val_loss: 8.9643e-07
Epoch 20/50
1311/1311 [==============================] - 49s 38ms/step - loss: 3.5034e-06 - val_loss: 1.6819e-06
Epoch 21/50
1311/1311 [==============================] - 45s 35ms/step - loss: 2.5422e-06 - val_loss: 2.9584e-06
Epoch 22/50
1311/1311 [==============================] - 42s 32ms/step - loss: 4.4201e-06 - val_loss: 1.7950e-06
Epoch 23/50
1311/1311 [==============================] - 41s 32ms/step - loss: 2.4149e-06 - val_loss: 3.5397e-06
Epoch 24/50
1311/1311 [==============================] - 41s 32ms/step - loss: 2.4425e-06 - val_loss: 1.1264e-06
Epoch 25/50
1311/1311 [==============================] - 42s 32ms/step - loss: 2.3470e-06 - val_loss: 1.5761e-06
Epoch 26/50
1311/1311 [==============================] - 41s 31ms/step - loss: 3.3967e-06 - val_loss: 8.4410e-06
Epoch 27/50
1311/1311 [==============================] - 44s 33ms/step - loss: 2.1801e-06 - val_loss: 9.3589e-07
Epoch 28/50
1311/1311 [==============================] - 48s 37ms/step - loss: 2.1435e-06 - val_loss: 1.2334e-06
Epoch 29/50
1311/1311 [==============================] - 61s 46ms/step - loss: 1.8684e-06 - val_loss: 5.5123e-07
Epoch 30/50
1311/1311 [==============================] - 40s 31ms/step - loss: 1.9194e-06 - val_loss: 1.2430e-06
Epoch 31/50
1311/1311 [==============================] - 42s 32ms/step - loss: 1.9218e-06 - val_loss: 2.3787e-06
Epoch 32/50
1311/1311 [==============================] - 50s 38ms/step - loss: 2.1716e-06 - val_loss: 1.1628e-06
Epoch 33/50
1311/1311 [==============================] - 45s 35ms/step - loss: 1.8156e-06 - val_loss: 1.2759e-06
Epoch 34/50
1311/1311 [==============================] - 57s 43ms/step - loss: 1.6195e-06 - val_loss: 3.1208e-06
Epoch 35/50
1311/1311 [==============================] - 50s 38ms/step - loss: 3.4217e-06 - val_loss: 1.2604e-06
Epoch 36/50
1311/1311 [==============================] - 48s 36ms/step - loss: 1.5486e-06 - val_loss: 2.1367e-06
Epoch 37/50
1311/1311 [==============================] - 43s 33ms/step - loss: 1.5729e-06 - val_loss: 1.6547e-06
Epoch 38/50
1311/1311 [==============================] - 51s 39ms/step - loss: 1.4122e-06 - val_loss: 1.1133e-06
Epoch 39/50
1311/1311 [==============================] - 46s 35ms/step - loss: 1.4515e-06 - val_loss: 1.1246e-06
Epoch 40/50
1311/1311 [==============================] - 43s 32ms/step - loss: 1.3622e-06 - val_loss: 3.3473e-06
Epoch 41/50
1311/1311 [==============================] - 44s 34ms/step - loss: 1.5986e-06 - val_loss: 2.9740e-06
Epoch 42/50
1311/1311 [==============================] - 46s 35ms/step - loss: 3.4352e-06 - val_loss: 7.7364e-07
Epoch 43/50
1311/1311 [==============================] - 39s 30ms/step - loss: 2.2802e-06 - val_loss: 1.0786e-06
Epoch 44/50
1311/1311 [==============================] - 39s 29ms/step - loss: 9.1918e-07 - val_loss: 1.1899e-06
Epoch 45/50
1311/1311 [==============================] - 46s 35ms/step - loss: 1.1258e-06 - val_loss: 6.7904e-07
Epoch 46/50
1311/1311 [==============================] - 47s 36ms/step - loss: 9.2028e-07 - val_loss: 1.4309e-06
Epoch 47/50
1311/1311 [==============================] - 44s 34ms/step - loss: 1.8474e-06 - val_loss: 9.7508e-07
Epoch 48/50
1311/1311 [==============================] - 43s 33ms/step - loss: 1.2926e-06 - val_loss: 1.1840e-06
Epoch 49/50
1311/1311 [==============================] - 43s 33ms/step - loss: 9.2501e-07 - val_loss: 1.2624e-06
Epoch 50/50
1311/1311 [==============================] - 44s 34ms/step - loss: 1.0491e-06 - val_loss: 1.4117e-06
```

Analyze the training and validation losses to assess the model's learning process.

In [64]:
```python
# we are Plotting the training and validation loss
plt.plot(rnn_history.history['loss'], label='Training Loss')
plt.plot(rnn_history.history['val_loss'], label='Validation Loss')
plt.title('RNN Autoencoder Training and Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.show()
```



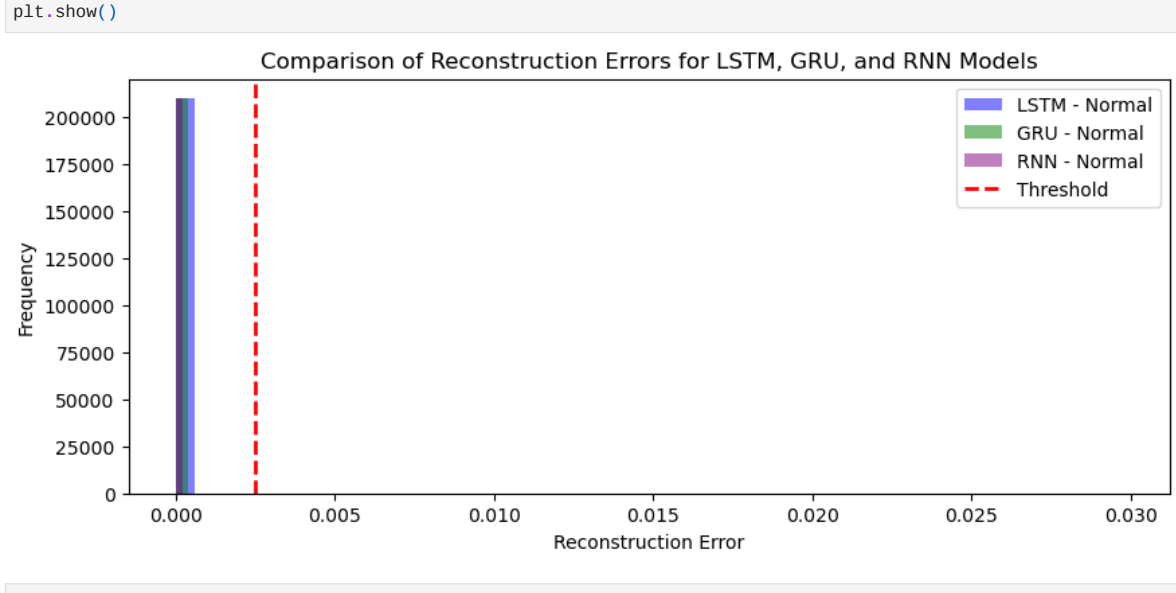
In [65]:
```python
# we Predict the reconstruction from each model
lstm_predictions = autoencoder.predict(X_test)
gru_predictions = gru_autoencoder.predict(X_test)
rnn_predictions = rnn_autoencoder.predict(X_test)
```

```
6554/6554 [==============================] - 35s 5ms/step
6554/6554 [==============================] - 33s 5ms/step
6554/6554 [==============================] - 19s 3ms/step
```

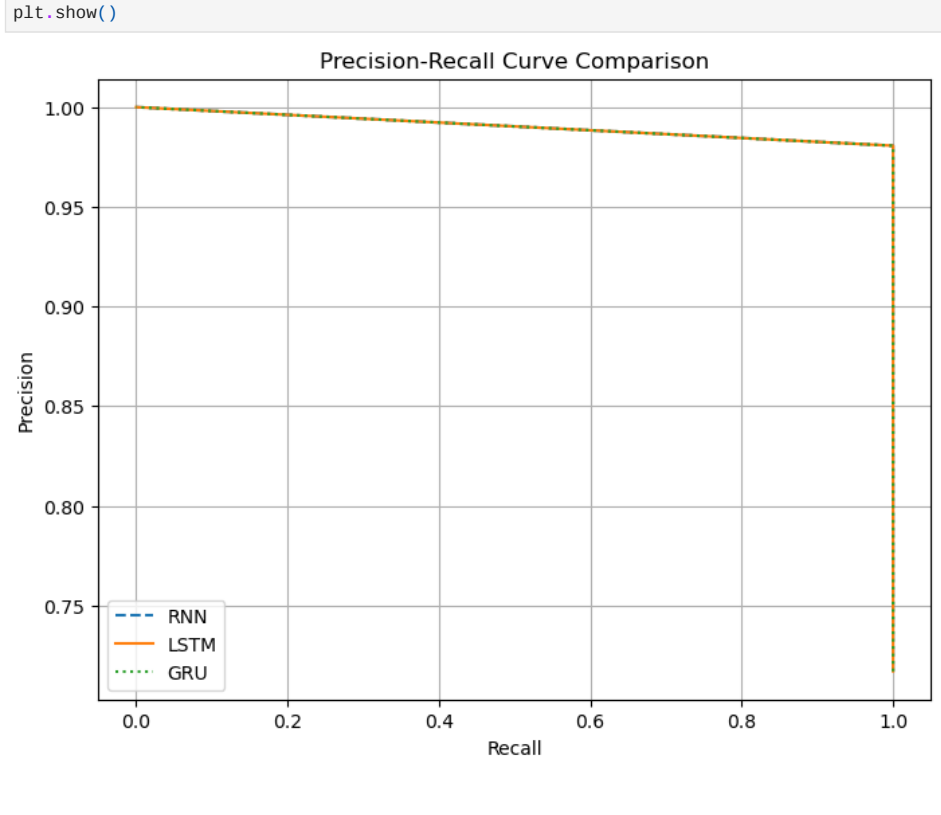
In [66]:
```python
# we Calculate reconstruction error for each model
lstm_reconstruction_error = np.mean(np.power(X_test - lstm_predictions, 2), axis=(1, 2))
gru_reconstruction_error = np.mean(np.power(X_test - gru_predictions, 2), axis=(1, 2))
rnn_reconstruction_error = np.mean(np.power(X_test - rnn_predictions, 2), axis=(1, 2))
```

In [67]:
```python
# we Define threshold based on the LSTM model
threshold = np.percentile(lstm_reconstruction_error, 95)
```

In [80]:
```python
# we are Plotting the histograms for comparison
plt.figure(figsize=(10, 4))
plt.hist(lstm_reconstruction_error, bins=50, color='blue', alpha=0.5, label='LSTM - Normal')
plt.hist(gru_reconstruction_error, bins=50, color='green', alpha=0.5, label='GRU - Normal')
plt.hist(rnn_reconstruction_error, bins=50, color='purple', alpha=0.5, label='RNN - Normal')
plt.axvline(threshold, color='red', linestyle='dashed', linewidth=2, label='Threshold')
plt.title('Comparison of Reconstruction Errors for LSTM, GRU, and RNN Models')
plt.xlabel('Reconstruction Error')
plt.ylabel('Frequency')
plt.legend()
plt.show()
```



In [192…
```python
precision_rnn, recall_rnn, _ = precision_recall_curve(y_true, y_pred_rnn)
precision_lstm, recall_lstm, _ = precision_recall_curve(y_true, y_pred_lstm)
precision_gru, recall_gru, _ = precision_recall_curve(y_true, y_pred_gru)

plt.figure(figsize=(8, 6))
plt.plot(recall_rnn, precision_rnn, label='RNN', linestyle='--')
plt.plot(recall_lstm, precision_lstm, label='LSTM', linestyle='-')
plt.plot(recall_gru, precision_gru, label='GRU', linestyle=':')
plt.xlabel('Recall')
plt.ylabel('Precision')
plt.title('Precision-Recall Curve Comparison')
plt.legend()
plt.grid()
plt.show()
```



LSTM - Testing the Trained Network on both normal and anomalous data with the same temporal window size to evaluate the model's performance.

In [206…
```python
normal_sequences = create_sequences(normal_data_scaled, window_size)

anomalous_sequences = create_sequences(anomalous_data_scaled, window_size)


X_train, X_test = train_test_split(normal_sequences, test_size=0.2, random_state=42)
```

In [207…
```python
# Our autoencoder is a trained model from the original code.
# We can Predict the test sequences

normal_predictions = autoencoder.predict(X_test)
anomalous_predictions = autoencoder.predict(anomalous_sequences)
```

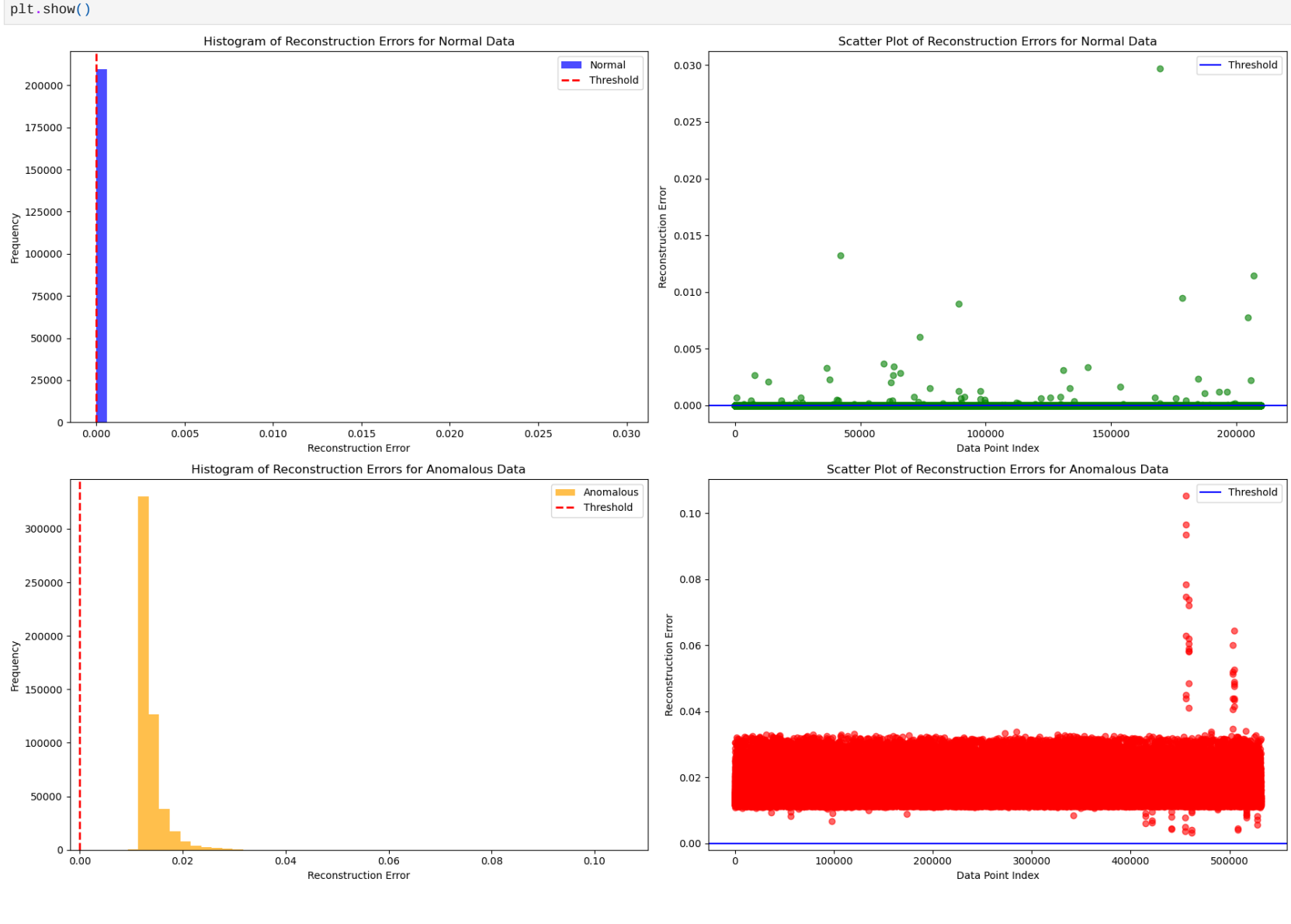
```
6554/6554 [==============================] - 40s 6ms/step
16606/16606 [==============================] - 101s 6ms/step
```

In [208…
```python
# We can Calculate reconstruction error

normal_reconstruction_error = np.mean(np.power(X_test - normal_predictions, 2), axis=(1, 2))
anomalous_reconstruction_error = np.mean(np.power(anomalous_sequences - anomalous_predictions, 2), axis=(1, 2))
```

In [209…
```python
# Defination threshold for anomaly detection

threshold = np.percentile(normal_reconstruction_error, 95)

# Classifying sequences as normal or anomalous
normal_classified_as_anomalous = normal_reconstruction_error >= threshold
anomalous_classified_as_normal = anomalous_reconstruction_error < threshold
```

Visualizing Reconstruction Errors and Thresholds for Anomaly Detection on Normal and Anomalous Data"

In [210…
```python
fig, axs = plt.subplots(2, 2, figsize=(18, 12))

axs[0, 0].hist(normal_reconstruction_error, bins=50, alpha=0.7, color='blue', label='Normal')
axs[0, 0].axvline(threshold, color='red', linestyle='dashed', linewidth=2, label='Threshold')
axs[0, 0].set_title('Histogram of Reconstruction Errors for Normal Data')
axs[0, 0].set_xlabel('Reconstruction Error')
axs[0, 0].set_ylabel('Frequency')
axs[0, 0].legend()

axs[0, 1].scatter(range(len(normal_reconstruction_error)), normal_reconstruction_error, color='green', alpha=0.6)
axs[0, 1].axhline(y=threshold, color='blue', linestyle='-', label='Threshold')
axs[0, 1].set_title('Scatter Plot of Reconstruction Errors for Normal Data')
axs[0, 1].set_xlabel('Data Point Index')
axs[0, 1].set_ylabel('Reconstruction Error')
axs[0, 1].legend()

axs[1, 0].hist(anomalous_reconstruction_error, bins=50, alpha=0.7, color='orange', label='Anomalous')
axs[1, 0].axvline(threshold, color='red', linestyle='dashed', linewidth=2, label='Threshold')
axs[1, 0].set_title('Histogram of Reconstruction Errors for Anomalous Data')
axs[1, 0].set_xlabel('Reconstruction Error')
axs[1, 0].set_ylabel('Frequency')
axs[1, 0].legend()

axs[1, 1].scatter(range(len(anomalous_reconstruction_error)), anomalous_reconstruction_error, color='red', alpha=0.6)
axs[1, 1].axhline(y=threshold, color='blue', linestyle='-', label='Threshold')
axs[1, 1].set_title('Scatter Plot of Reconstruction Errors for Anomalous Data')
axs[1, 1].set_xlabel('Data Point Index')
axs[1, 1].set_ylabel('Reconstruction Error')
axs[1, 1].legend()

plt.tight_layout()
plt.show()
```



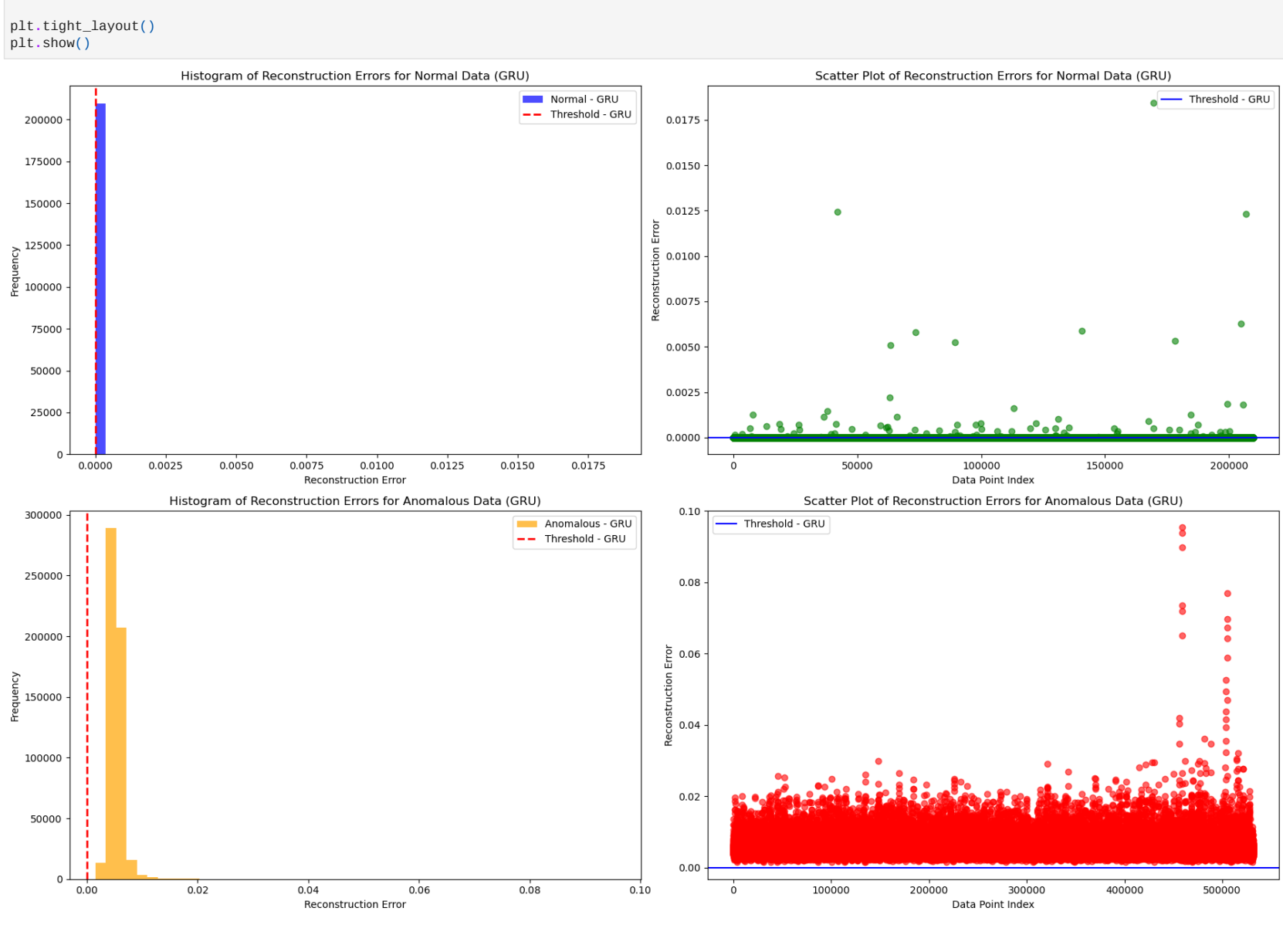
Testing on Normal and Anomalous Sequences - Anomaly Detection and Classification Using GRU Autoencoder with Thresholding

In [84]:
```python
# GRU model predictions on test sequences
gru_normal_predictions = gru_autoencoder.predict(X_test)
gru_anomalous_predictions = gru_autoencoder.predict(anomalous_sequences)

# Calculate reconstruction error for GRU model
gru_normal_reconstruction_error = np.mean(np.power(X_test - gru_normal_predictions, 2), axis=(1, 2))
gru_anomalous_reconstruction_error = np.mean(np.power(anomalous_sequences - gru_anomalous_predictions, 2), axis=(1, 2))

#  threshold for anomaly detection based on the GRU model
gru_threshold = np.percentile(gru_normal_reconstruction_error, 95)

# sequences as normal or anomalous based on GRU model
gru_normal_classified_as_anomalous = gru_normal_reconstruction_error >= gru_threshold
gru_anomalous_classified_as_normal = gru_anomalous_reconstruction_error < gru_threshold
```

```
6554/6554 [==============================] - 30s 5ms/step
16606/16606 [==============================] - 87s 5ms/step
```

Visualizing Reconstruction Errors and Thresholds for GRU-Based Anomaly Detection on Normal and Anomalous Data

In [86]:
```python
fig, axs = plt.subplots(2, 2, figsize=(18, 12))

axs[0, 0].hist(gru_normal_reconstruction_error, bins=50, alpha=0.7, color='blue', label='Normal - GRU')
axs[0, 0].axvline(gru_threshold, color='red', linestyle='dashed', linewidth=2, label='Threshold - GRU')
axs[0, 0].set_title('Histogram of Reconstruction Errors for Normal Data (GRU)')
axs[0, 0].set_xlabel('Reconstruction Error')
axs[0, 0].set_ylabel('Frequency')
axs[0, 0].legend()

axs[0, 1].scatter(range(len(gru_normal_reconstruction_error)), gru_normal_reconstruction_error, color='green', alpha=0.6)
axs[0, 1].axhline(y=gru_threshold, color='blue', linestyle='-', label='Threshold - GRU')
axs[0, 1].set_title('Scatter Plot of Reconstruction Errors for Normal Data (GRU)')
axs[0, 1].set_xlabel('Data Point Index')
axs[0, 1].set_ylabel('Reconstruction Error')
axs[0, 1].legend()

axs[1, 0].hist(gru_anomalous_reconstruction_error, bins=50, alpha=0.7, color='orange', label='Anomalous - GRU')
axs[1, 0].axvline(gru_threshold, color='red', linestyle='dashed', linewidth=2, label='Threshold - GRU')
axs[1, 0].set_title('Histogram of Reconstruction Errors for Anomalous Data (GRU)')
axs[1, 0].set_xlabel('Reconstruction Error')
axs[1, 0].set_ylabel('Frequency')
axs[1, 0].legend()

axs[1, 1].scatter(range(len(gru_anomalous_reconstruction_error)), gru_anomalous_reconstruction_error, color='red', alpha=0.6)
axs[1, 1].axhline(y=gru_threshold, color='blue', linestyle='-', label='Threshold - GRU')
axs[1, 1].set_title('Scatter Plot of Reconstruction Errors for Anomalous Data (GRU)')
axs[1, 1].set_xlabel('Data Point Index')
axs[1, 1].set_ylabel('Reconstruction Error')
axs[1, 1].legend()

plt.tight_layout()
plt.show()
```



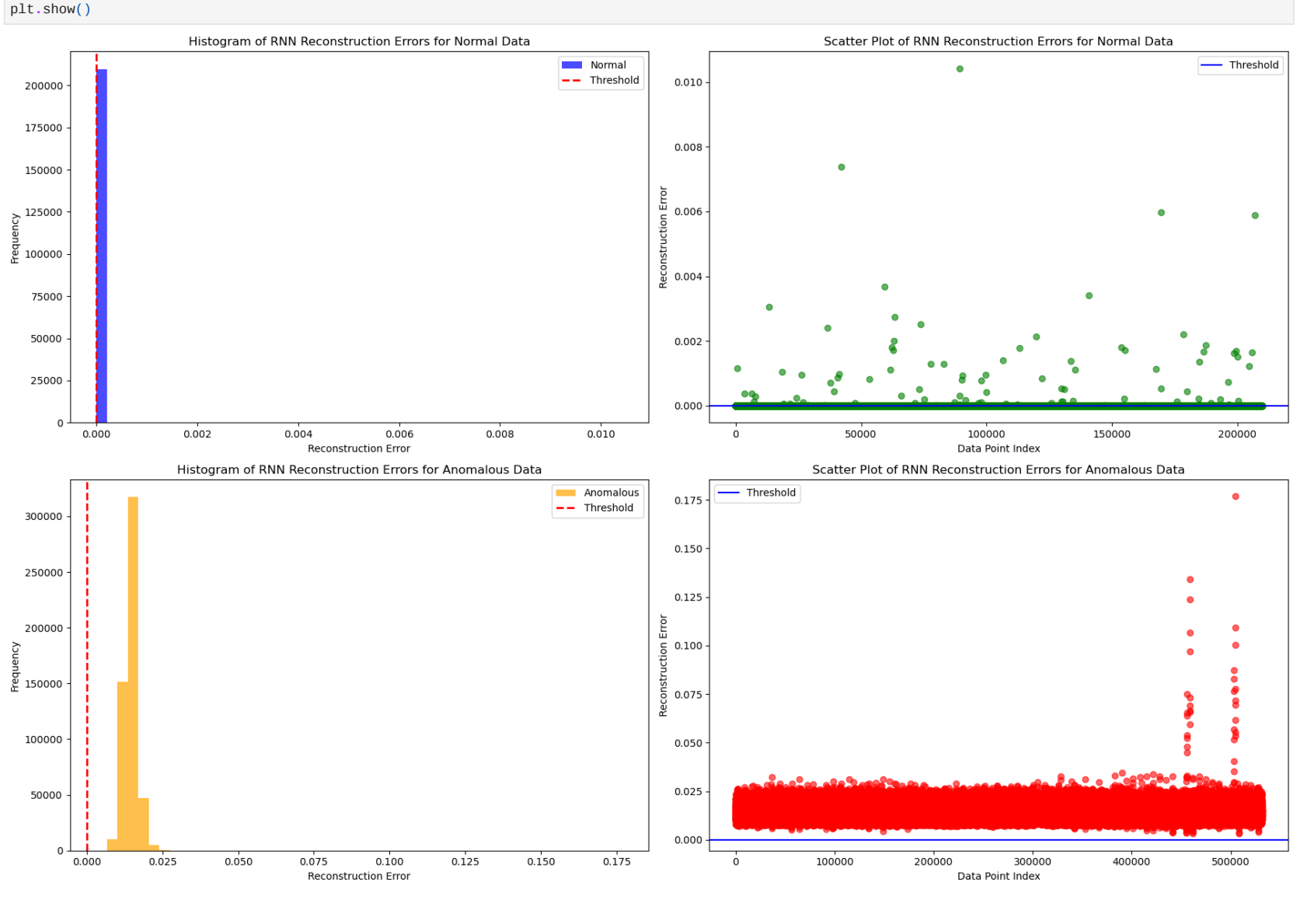
Testing on Normal and Anomalous Sequences - Anomaly Detection and Classification Using RNN Autoencoder with Thresholding

In [85]:
```python
rnn_normal_predictions = rnn_autoencoder.predict(X_test)
rnn_anomalous_predictions = rnn_autoencoder.predict(anomalous_sequences)

# we Calculat reconstruction error for RNN model
rnn_normal_reconstruction_error = np.mean(np.power(X_test - rnn_normal_predictions, 2), axis=(1, 2))
rnn_anomalous_reconstruction_error = np.mean(np.power(anomalous_sequences - rnn_anomalous_predictions, 2), axis=(1, 2))

# we Define threshold for anomaly detection based on the RNN model
rnn_threshold = np.percentile(rnn_normal_reconstruction_error, 95)

# we Classify sequences as normal or anomalous based on RNN model
rnn_normal_classified_as_anomalous = rnn_normal_reconstruction_error >= rnn_threshold
rnn_anomalous_classified_as_normal = rnn_anomalous_reconstruction_error < rnn_threshold
```

```
6554/6554 [==============================] - 18s 3ms/step
16606/16606 [==============================] - 50s 3ms/step
```

Visualizing RNN Autoencoder Reconstruction Errors and Thresholds for Anomaly Detection on Normal and Anomalous Data

In [133…
```python
fig, axs = plt.subplots(2, 2, figsize=(18, 12))


axs[0, 0].hist(rnn_normal_reconstruction_error, bins=50, alpha=0.7, color='blue', label='Normal')
axs[0, 0].axvline(threshold, color='red', linestyle='dashed', linewidth=2, label='Threshold')
axs[0, 0].set_title('Histogram of RNN Reconstruction Errors for Normal Data')
axs[0, 0].set_xlabel('Reconstruction Error')
axs[0, 0].set_ylabel('Frequency')
axs[0, 0].legend()


axs[0, 1].scatter(range(len(rnn_normal_reconstruction_error)), rnn_normal_reconstruction_error, color='green', alpha=0.6)
axs[0, 1].axhline(y=threshold, color='blue', linestyle='-', label='Threshold')
axs[0, 1].set_title('Scatter Plot of RNN Reconstruction Errors for Normal Data')
axs[0, 1].set_xlabel('Data Point Index')
axs[0, 1].set_ylabel('Reconstruction Error')
axs[0, 1].legend()


axs[1, 0].hist(rnn_anomalous_reconstruction_error, bins=50, alpha=0.7, color='orange', label='Anomalous')
axs[1, 0].axvline(threshold, color='red', linestyle='dashed', linewidth=2, label='Threshold')
axs[1, 0].set_title('Histogram of RNN Reconstruction Errors for Anomalous Data')
axs[1, 0].set_xlabel('Reconstruction Error')
axs[1, 0].set_ylabel('Frequency')
axs[1, 0].legend()

axs[1, 1].scatter(range(len(rnn_anomalous_reconstruction_error)), rnn_anomalous_reconstruction_error, color='red', alpha=0.6)
axs[1, 1].axhline(y=threshold, color='blue', linestyle='-', label='Threshold')
axs[1, 1].set_title('Scatter Plot of RNN Reconstruction Errors for Anomalous Data')
axs[1, 1].set_xlabel('Data Point Index')
axs[1, 1].set_ylabel('Reconstruction Error')
axs[1, 1].legend()

plt.tight_layout()
plt.show()
```



Determine key performance metrics such as accuracy, precision, recall, and F1 score to gauge the classification's effectiveness.

In [156…
```python
def calculate_performance_metrics(true_labels, predicted_labels):
    accuracy = accuracy_score(true_labels, predicted_labels)
    precision = precision_score(true_labels, predicted_labels)
    recall = recall_score(true_labels, predicted_labels)
    f1 = f1_score(true_labels, predicted_labels)
    return accuracy, precision, recall, f1
```

In [157…
```python
true_labels_normal = np.zeros(len(normal_reconstruction_error))
true_labels_anomalous = np.ones(len(anomalous_reconstruction_error))
true_labels = np.concatenate([true_labels_normal, true_labels_anomalous])
```

In [158…
```python
def classify_as_normal(errors, threshold):
    return (errors < threshold).astype(int)

def classify_as_anomalous(errors, threshold):
    return (errors >= threshold).astype(int)
```

In [159…
```python
predicted_labels_lstm = np.concatenate([
    classify_as_normal(normal_reconstruction_error, threshold),
    classify_as_anomalous(anomalous_reconstruction_error, threshold)
])
accuracy_lstm, precision_lstm, recall_lstm, f1_score_lstm = calculate_performance_metrics(true_labels, predicted_labels_lstm)
print("LSTM Model Performance Metrics:")
print(f"Accuracy: {accuracy_lstm:.4f}, Precision: {precision_lstm:.4f}, Recall: {recall_lstm:.4f}, F1 Score: {f1_score_lstm:.4f}\n")
```

```
LSTM Model Performance Metrics:
Accuracy: 0.7312, Precision: 0.7273, Recall: 1.0000, F1 Score: 0.8421
```

In [160…
```python
predicted_labels_gru = np.concatenate([
    classify_as_normal(gru_normal_reconstruction_error, gru_threshold),
    classify_as_anomalous(gru_anomalous_reconstruction_error, gru_threshold)
])
accuracy_gru, precision_gru, recall_gru, f1_score_gru = calculate_performance_metrics(true_labels, predicted_labels_gru)
print("GRU Model Performance Metrics:")
print(f"Accuracy: {accuracy_gru:.4f}, Precision: {precision_gru:.4f}, Recall: {recall_gru:.4f}, F1 Score: {f1_score_gru:.4f}\n")
```

```
GRU Model Performance Metrics:
Accuracy: 0.7312, Precision: 0.7273, Recall: 1.0000, F1 Score: 0.8421
```

In [161…
```python
predicted_labels_rnn = np.concatenate([
    classify_as_normal(rnn_normal_reconstruction_error, rnn_threshold),
    classify_as_anomalous(rnn_anomalous_reconstruction_error, rnn_threshold)
])
accuracy_rnn, precision_rnn, recall_rnn, f1_score_rnn = calculate_performance_metrics(true_labels, predicted_labels_rnn)
print("RNN Model Performance Metrics:")
print(f"Accuracy: {accuracy_rnn:.4f}, Precision: {precision_rnn:.4f}, Recall: {recall_rnn:.4f}, F1 Score: {f1_score_rnn:.4f}\n")
```

```
RNN Model Performance Metrics:
Accuracy: 0.7312, Precision: 0.7273, Recall: 1.0000, F1 Score: 0.8421
```

In [162…
```python
def plot_performance_metrics(metrics_lstm, metrics_gru, metrics_rnn, labels):
    x = np.arange(len(labels))
    width = 0.2

    fig, ax = plt.subplots()
    ax.bar(x - width, metrics_lstm, width, label='LSTM')
    ax.bar(x, metrics_gru, width, label='GRU')
    ax.bar(x + width, metrics_rnn, width, label='RNN')

    ax.set_ylabel('Scores')
    ax.set_title('Performance Metrics by Model')
    ax.set_xticks(x)
    ax.set_xticklabels(labels)
    ax.legend()
    plt.show()

metrics_lstm = [accuracy_lstm, precision_lstm, recall_lstm, f1_score_lstm]
metrics_gru = [accuracy_gru, precision_gru, recall_gru, f1_score_gru]
metrics_rnn = [accuracy_rnn, precision_rnn, recall_rnn, f1_score_rnn]
labels = ['Accuracy', 'Precision', 'Recall', 'F1 Score']

plot_performance_metrics(metrics_lstm, metrics_gru, metrics_rnn, labels)
```